

CFD 2025-26 – Lab 08 – solution

The full code for this lab is provided in the notebook `CFDlab08.ipynb`.

Exercise 1

In the mesh `elbow3.msh`, the tag 1 identifies the inlet, 2 the outlet, and 3-4 the pipe walls. This tagging is fine for the imposition of no-slip conditions (case (i)), whereas for free-slip conditions we need to distinguish between horizontal walls where $\mathbf{n} = \pm \mathbf{j}$, $\mathbf{t} = \pm \mathbf{i}$, and vertical ones where $\mathbf{n} = \pm \mathbf{i}$, $\mathbf{t} = \pm \mathbf{j}$: this possibility is provided by `elbow3bis.msh`, with tags that can be inspected by `tripplot`.

- 1 We define the following spaces and forms:

$$\begin{aligned} V_D^{(i)} &= \{\mathbf{v} \in [H^1(\Omega)]^2 \text{ s.t. } \mathbf{v} = \mathbf{u}_{\text{in}} \text{ on } \Gamma_{\text{in}}, \mathbf{u} = \mathbf{0} \text{ on } \Gamma_{\text{wall}}\}, \\ V_D^{(ii)} &= \{\mathbf{v} \in [H^1(\Omega)]^2 \text{ s.t. } \mathbf{v} = \mathbf{u}_{\text{in}} \text{ on } \Gamma_{\text{in}}, \mathbf{u} \cdot \mathbf{n} = 0 \text{ on } \Gamma_{\text{wall}}\}, \\ V_0^{(i)} &= [H_{\partial\Omega \setminus \Gamma_{\text{out}}}^1]^2, \\ V_0^{(ii)} &= \{\mathbf{v} \in [H_{\Gamma_{\text{in}}}^1(\Omega)]^2 \text{ s.t. } \mathbf{u} \cdot \mathbf{n} = 0 \text{ on } \Gamma_{\text{wall}}\}, \\ Q &= L^2(\Omega), \end{aligned}$$

$$a(\mathbf{u}, \mathbf{v}) = \int_{\Omega} \nabla \mathbf{u} : \nabla \mathbf{v} \, dx, \quad b(\mathbf{v}, q) = - \int_{\Omega} \operatorname{div} \mathbf{v} \, q \, dx, \quad c(\mathbf{w}, \mathbf{u}, \mathbf{v}) = \int_{\Omega} (\mathbf{w} \cdot \nabla) \mathbf{u} \cdot \mathbf{v} \, dx,$$

where $\mathbf{u}_{\text{in}}(x, y) = 4y(1 - y)\mathbf{i}$.

Let $\mathcal{R}^{(i)} \mathbf{u}_{\text{in}} \in V_D^{(i)}$ be a lifting of the inlet velocity, and let us assume it to be solenoidal (i.e. $\operatorname{div} \mathcal{R}^{(i)} \mathbf{u}_{\text{in}} = 0$). Proceeding similarly to the previous labs and with the usual notation, we can obtain the following (nonlinear) weak form of the problem with *no-slip* wall conditions:

$$\begin{aligned} \text{find } \mathbf{u} = \tilde{\mathbf{u}} + \mathcal{R}^{(i)} \mathbf{u}_{\text{in}} \in V_D^{(i)}, \text{ with } \tilde{\mathbf{u}} \in V_0^{(i)}, \text{ and } p \in Q \text{ such that} \\ G(\tilde{\mathbf{u}}, p; \mathcal{R}^{(i)} \mathbf{u}_{\text{in}})[\mathbf{v}, q] = 0 \quad \forall (\mathbf{v}, q) \in V_0^{(i)} \times Q, \end{aligned} \quad (1)$$

where, for fixed $\tilde{\mathbf{u}}, p, \mathcal{R}^{(i)} \mathbf{u}_{\text{in}}$, we define the residual functional $G(\tilde{\mathbf{u}}, p; \mathcal{R}^{(i)} \mathbf{u}_{\text{in}}) : [H^1(\Omega)]^2 \times Q \rightarrow \mathbb{R}$ as follows:

$$\begin{aligned} G(\tilde{\mathbf{u}}, p; \mathcal{R}^{(i)} \mathbf{u}_{\text{in}})[\mathbf{v}, q] &= a(\tilde{\mathbf{u}}, \mathbf{v}) + c(\tilde{\mathbf{u}}, \tilde{\mathbf{u}}, \mathbf{v}) + b(\mathbf{v}, p) - b(\tilde{\mathbf{u}}, q) \\ &\quad - a(\mathcal{R}^{(i)} \mathbf{u}_{\text{in}}, \mathbf{v}) - c(\tilde{\mathbf{u}}, \mathcal{R}^{(i)} \mathbf{u}_{\text{in}}, \mathbf{v}) - c(\mathcal{R}^{(i)} \mathbf{u}_{\text{in}}, \tilde{\mathbf{u}}, \mathbf{v}) - c(\mathcal{R}^{(i)} \mathbf{u}_{\text{in}}, \mathcal{R}^{(i)} \mathbf{u}_{\text{in}}, \mathbf{v}). \end{aligned} \quad (2)$$

In the case of *free-slip* conditions, the boundary terms coming from testing the momentum equation and integrating by parts can be treated as follows:

$$\begin{aligned} \int_{\partial\Omega} (\nu \nabla \mathbf{u} - pI) \mathbf{n} \cdot \mathbf{v} &= \int_{\Gamma_{\text{in}}} (\nu \nabla \mathbf{u} - pI) \mathbf{n} \cdot \mathbf{x} + \int_{\Gamma_{\text{out}}} (\nu \nabla \mathbf{u} - pI) \mathbf{n} \cdot \mathbf{v} + \int_{\Gamma_{\text{wall}}} (\nu \nabla \mathbf{u} - pI) \mathbf{n} \cdot \mathbf{v} \\ &= \int_{\Gamma_{\text{wall}}} (\nu \nabla \mathbf{u} - pI) \mathbf{n} \cdot (\mathbf{v} \cdot \mathbf{n}) \mathbf{n} + \int_{\Gamma_{\text{wall}}} (\nu \nabla \mathbf{u} - pI) \mathbf{n} \cdot (\mathbf{v} \cdot \mathbf{t}) \mathbf{t} \\ &= \int_{\Gamma_{\text{wall}}} (\nu \nabla \mathbf{u} - pI) \mathbf{n} \cdot \mathbf{t} (\mathbf{v} \cdot \mathbf{t}) = 0, \end{aligned}$$

where we have used that, for any vector $\mathbf{w} \in \mathbb{R}^2$, $\mathbf{w} = (\mathbf{w} \cdot \mathbf{n})\mathbf{n} + (\mathbf{w} \cdot \mathbf{t})\mathbf{t}$, and the striked-out terms vanish due to the boundary conditions. Then, introducing a solenoidal lifting $\mathcal{R}^{(ii)}\mathbf{u}_{\text{in}} \in V_D^{(ii)}$, the nonlinear weak problem of case (ii) reads as follows:

$$\begin{aligned} \text{find } \mathbf{u} = \check{\mathbf{u}} + \mathcal{R}^{(ii)}\mathbf{u}_{\text{in}} \in V_D^{(ii)}, \text{ with } \check{\mathbf{u}} \in V_0^{(ii)}, \text{ and } p \in Q \text{ such that} \\ G(\check{\mathbf{u}}, p; \mathcal{R}^{(ii)}\mathbf{u}_{\text{in}})[\mathbf{v}, q] = 0 \quad \forall (\mathbf{v}, q) \in V_0^{(ii)} \times Q, \end{aligned} \quad (3)$$

where the residual functional $G(\check{\mathbf{u}}, p; \mathcal{R}^{(ii)}\mathbf{u}_{\text{in}})$ is defined analogously to (2).

- 2 Firedrake’s command `solve`, that we have used as linear solver in previous labs, can be used also to solve nonlinear problems, using the following syntax:

```
solve(G == 0, wh, bcs=bcs, solver_parameters=param)
```

where $\mathbf{G}$ is a suitable implementation of the residual G introduced in (2), while the other arguments are analogous to the case of a linear solver. Notice that, for the purpose of Firedrake implementation, the residual functional G has the “same” definition in both cases (i) and (ii), since Dirichlet boundary conditions are enforced by `DirichletBC` objects (passed to `solve` by `bcs`) and thus we do not need to include in G the terms depending on the lifting. The structure of $\mathbf{G}$ is explained in the documentation of the function `nonlinear_problem_residual` provided in the template code. This function depends on the data and on a specific instance of the discrete solution $(\mathbf{uh}, \mathbf{ph})$, and returns a *linear form*, i.e. a functional over test functions: indeed, $\mathbf{uh}$ and $\mathbf{ph}$ must be components of an actual `Function`, while $\mathbf{v}$ and $\mathbf{p}$ are “placeholders” created by `TestFunctions` (see the cell *Assembly of the residual*).

split vs. subfunctions

In Firedrake, there are (at least) two different ways of extracting components of a given `Function` `wh` (that we assume here to have 2 components: velocity and pressure). The command `wh.subfunctions`, which we have used in the previous labs, returns two *views* of the components that can almost completely be used as actual `Functions` of their own. Thus, `subfunctions` is the method of choice if we want to extract the components of the solution to post-process it, e.g. to plot or to compute error norms: see the post-processing part of `CFDlab08.ipynb`. The command `split(wh)`, instead, returns two “weaker” views of `wh`, that cannot be used for post-processing. However, they are necessary to properly define a residual functional such as $\mathbf{G}$: see cell *Assembly of the residual*.

To solve the equation defined by the residual $\mathbf{G}$, we can use different algorithms, which can be selected and configured in the parameters passed to the solver. In this course we use the *Scalable Nonlinear Equations Solvers* (SNES) of PETSc, whose documentation and list of options is provided at <https://petsc.org/release/manual/snes/>. We set-up the solver to use the Newton method by `'snes_type': 'newtonls'` (standing for Newton *with Line Search*), and other options are commented in the template code.

The stream function problem is a Poisson problem with the vorticity ω as a source term. Notice that the boundary conditions of the stream problem are chosen to be compatible

with the velocity field of the Navier-Stokes problem. Indeed, at the inlet Γ_{in} ,

$$\begin{aligned} \mathbf{u}(0, y) &= 4y(1 - y)\mathbf{i} \quad \Rightarrow \quad \omega(0, y) = 0 - (4 - 8y) = 8y - 4 \\ \Rightarrow \quad \psi(0, y) &= - \int_0^y \left(\int_0^{\tilde{y}} \omega(0, \tilde{y}) d\tilde{y} \right) d\hat{y} = \int_0^y [-4\tilde{y}^2 + 4\tilde{y}]_0^{\hat{y}} d\hat{y} = \left[-\frac{4}{3}\hat{y}^3 + 2\hat{y}^2 \right]_0^y \\ \Rightarrow \quad \psi(0, y) &= 2y^2 - \frac{4}{3}y^3 \rightarrow 2/3 \text{ for } y \rightarrow 1. \end{aligned}$$

To define the discrete vorticity ω_h corresponding to the FE solution of the Navier-Stokes problem, we can use the component-wise derivative definition $\omega_h = \partial_x(\mathbf{u}_h)_y - \partial_y(\mathbf{u}_h)_x$, thanks to the operator `Dx(uh.sub(*), **)`, where `*` is the component of the function (e.g. 1 for $(\mathbf{u}_h)_y$) and `**` the direction of the derivative (e.g. 0 for ∂_x). Alternatively, UFL/Firedrake provide also the `rot` operator, that computes the expected quantity also in 2D (despite the online documentation states that it works only in 3D).

To pass to free-slip conditions, we only need to replace the object `bc_noslip` with conditions in which we set to 0 only *a component* of the velocity. Using the `sub` extractor recursively, by `W.sub(0).sub(0)/W.sub(0).sub(1)` we extract the velocity component of the mixed space (first `sub(0)`) and then the first/second component of such velocity. This is then set to 0 by a `scalar Constant(0)`, considering separately the vertical and horizontal portions of the walls, on which $\mathbf{n} = \pm\mathbf{i}$ and $\mathbf{n} = \pm\mathbf{j}$, respectively.

The results are imported in Paraview and then analyzed employing the *Glyph* and *Contour* filters. The Paraview state used to generate Figure 1 is provided in `stateLab8ex1.pvsm`. However, you cannot load this state as-is on your computer! If your solution files are named as `lab08_elbow3_freeslip_[velocity,pressure,stream].pvd`, you need to select the option *Search files under specified directory*, then click on `...` and navigate to the directory where your files are. If you have named them in a different way, you need to select the option *Choose file names* and then click on the `...` near each file name to navigate and choose your corresponding files.

- 3 The drag force and pressure jumps are defined in terms of integrals over portions of the boundary. As we have done in previous labs when computing error norms, we can use the `assemble` command to compute the integral of a form defined just over `Functions` or known data (namely where no `TrialFunctions` nor `TestFunctions` appear). In the case of drag, the boundary Γ is tagged as 33, and since $\mathbf{n} \perp \mathbf{i}$ pressure does not actually contribute to this force component. Therefore, the following two instructions are equivalent:

```
drag = assemble(
    -nu*inner(dot(grad(uh_out), normal), Constant((1.,0.))) * ds(33)
)
drag = assemble(-nu*(-Dx(uh_out.sub(0), 1)) * ds(33))
```

- 4 The results are reported in Figure 1 and in the following table:

	F_D	Δp
no slip	0.222465	1.491105
free slip	0.001657	0.578727

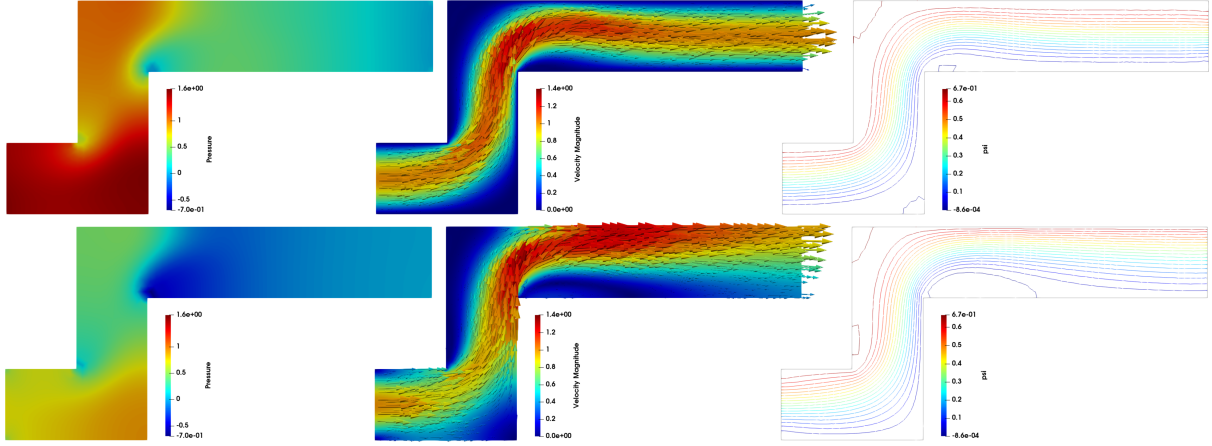


Figure 1: Exercises 1. Pressure (left), velocity (center) and streamlines (right) of the Navier-Stokes solution, enforcing no-slip (above) or free-slip (below) conditions on the pipe walls.

In terms of F_D , we can notice that the drag force is *positive*, meaning that the wall Γ is pushed *downstream* by the fluid. In the free-slip case, however, the force is way smaller (not exactly = 0 because the velocity profile is not exactly flat) than in the no-slip case, due to the absence of friction.

The overall pressure difference Δp between the input and output sections is smaller the free-slip case, meaning that the absence of friction reduces the amount of energy required to sustain the prescribed flowrate. We can notice that in the case of free-slip conditions, the inertia of the flow has a stronger effect on the velocity distribution: the flow seems to almost impinge on the wall (at least in the velocity plot) just after the two abrupt turns. Moreover, the outlet velocity profile is not parabolic, but more similar to a flat profile. The asymmetric effects of inertia can be seen also in the stream plots, particularly in the recirculation regions, which are significantly larger in the free slip case. In terms of pressure, we can notice smaller overall pressure differences in the free-slip case, associated to a smaller mechanical work to have the same flowrate in the pipe.

Exercise 2

We introduce the spaces

$$\begin{aligned} V_D &= \{\mathbf{v} \in [H_{\Gamma_{\text{cyl}}}^1(\Omega)]^2 \text{ s.t. } \mathbf{v} = \mathbf{u}_{\text{in}} \text{ on } \Gamma_{\text{in}}, \mathbf{u} \cdot \mathbf{n} = 0 \text{ on } \Gamma_{\text{wall}}\}, \\ V_0 &= \{\mathbf{v} \in [H_{\Gamma_{\text{in}} \cup \Gamma_{\text{cyl}}}^1(\Omega)]^2 \text{ s.t. } \mathbf{u} \cdot \mathbf{n} = 0 \text{ on } \Gamma_{\text{wall}}\}, \\ Q &= L^2(\Omega), \end{aligned}$$

where $\mathbf{u}_{\text{in}} = 1\mathbf{i}$, and the lifting $\mathcal{R}\mathbf{u}_{\text{in}}$. Then, the time-dependent solver basically consists in a time for-loop wrapped around the same nonlinear solver of Exercise 1: at each time step, we need to solve the nonlinear problem

$$\begin{aligned} \text{Find } (\mathbf{u}^{n+1}, p^{n+1}) &\in V_D \times Q \text{ such that } \mathbf{u}^{n+1} = \tilde{\mathbf{u}}^{n+1} + \mathcal{R}\mathbf{u}_{\text{in}}, \text{ with } \tilde{\mathbf{u}}^{n+1} \in V_0 \text{ and} \\ \mathcal{G}^{n+1}(\tilde{\mathbf{u}}^{n+1}, p^{n+1}; \mathcal{R}\mathbf{u}_{\text{in}})[\mathbf{v}, q] &= 0 \quad \forall (\mathbf{v}, q) \in V_0 \times Q, \end{aligned}$$

where the residual functional at time t^{n+1} is defined as:

$$\begin{aligned} \mathcal{G}^{n+1}(\check{\mathbf{u}}, p; \mathcal{R}\mathbf{u}_{\text{in}})[\mathbf{v}, q] \\ = G(\check{\mathbf{u}}, p; \mathcal{R}\mathbf{u}_{\text{in}})[\mathbf{v}, q] + \frac{1}{\Delta t} \int_{\Omega} \check{\mathbf{u}} \cdot \mathbf{v} \, d\mathbf{x} - \frac{1}{\Delta t} \int_{\Omega} \mathbf{u}^n \cdot \mathbf{v} \, d\mathbf{x} + \frac{1}{\Delta t} \int_{\Omega} \mathcal{R}\mathbf{u}_{\text{in}} \cdot \mathbf{v} \, d\mathbf{x}, \end{aligned} \quad (4)$$

being G the residual of the steady problem of Exercise 1. For further implementation details, we refer to the notebook `CFDlab08.ipynb`.